1. Data Retrieval, Integration & Storage

1.1 Data Retrieval

First of all, by calling the URL interface of the official Fuel API of NSW, obtain the key (Access Token) and add it to the request header. This key needs to be used as a header file in the subsequent URL as an interface for accessing service stations and fuel price data subsequently, that is, to obtain the real-time fuel price data of all gas stations and the basic information of gas stations through another fuel price interface URL. The specific implementation steps are as follows:

Step 1: Obtain the Access Token

Obtain the temporary access token (Access Token) required to access subsequent APIs by calling the Security URL provided by the official service. This request must include Authorization and Content-Type information in the request headers. The specific parameters are as follows:

Security URL: https://api.onegov.nsw.gov.au/oauth/client_credential/accesstoken

Method: GET

Headers: Authorization, Content-Type

Parameter: grant_type

The returned Access Token has an expiration limit, and all subsequent API requests must include this token in the request headers.

Step 2: Obtain information on gas stations and fuel prices

Using the obtained Access Token as part of the request header, access the URL provided in API version V1: https://api.onegov.nsw.gov.au/FuelPriceCheck/v1/fuel/prices. This URL contains real-time fuel price information for all service stations, along with detailed information for each station, including name, brand, address, and geographic coordinates. The data returned by the API is in JSON format and includes two key fields:

stations: Contains detailed information for each service station, such as station name, brand, address, station code, and geographic coordinates.

prices: Contains price information for different fuel types at each station, along with their latest update time.

The above request process is implemented using Python's requests library and returns data in JSON format.

Reason for Choosing the URL:

1. The basic Access Token is provided

2. Integrate the background information of fuel prices and gas stations to ensure the completeness of the obtained data, including all the core data required for the project.

3. Allows retrieval of all data with minimal number of access requests.

1.2 Data Processing

After obtaining the data, the returned JSON data needs to be processed, merged, and cleaned:

1. Processing Basic Information of Service Stations:

Extract the data in the stations field, store it in dictionary form and then convert it to DataFrame format. Each service station has a separate record, including basic information such as the brand, code, address, and latitude and longitude coordinates of the gas station.

2. Processing and Merging Fuel Price Data:

Since the data in the original "prices" field is in a long format (each record represents a gas station, a type of fuel product and its price), in order to facilitate the subsequent upload and saving of data, the data will be stored in dictionary format. For the same service station, we construct two fields, both in dictionary format:

fuel_price, formatted as {"U91":180,"U95":233...}

fuel_update_time, formatted as {"U91":"3/05/2025 23:25:02"...}

The prices and update times of different fuel types belonging to the same station are stored in the same dictionary, facilitating later storage and information upload.

3. Data Cleaning:

During data cleaning, it was observed that some service stations only contain the EV fuel type, with a price of 0 and the earliest update time dating back to 2022, suggesting these records are invalid. Therefore, a condition is added in the cleaning process: if a service station only has the EV fuel type with a price of 0, mark the record as invalid and remove it after setting the marker to False.

1.3 Data Integration

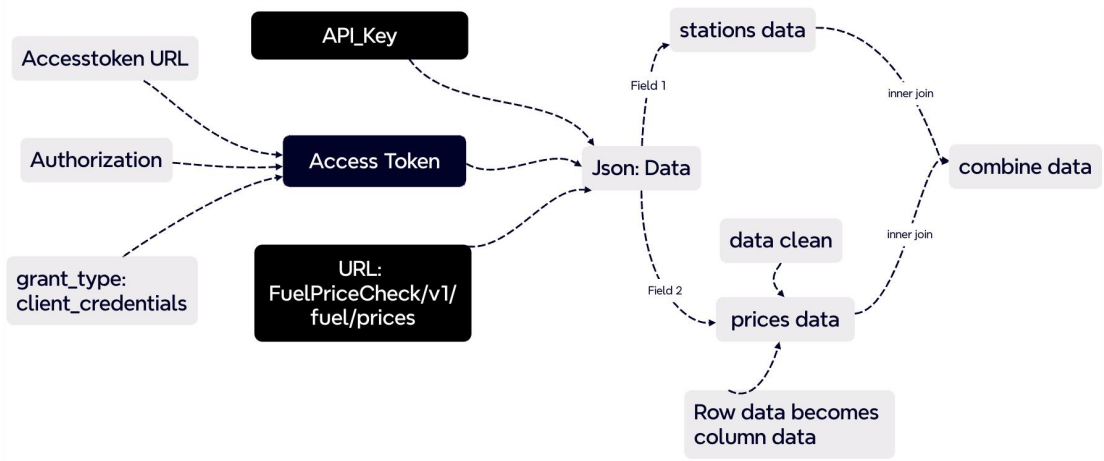
After data processing, the basic information of service stations and the fuel price data are joined via an inner join on the unique station code (stationcode), retaining same station information to form a unified dataset. The final integrated dataset contains 2,352 valid service station records, including detailed information and the latest fuel prices, satisfying real-time data display requirements.

The entire process is automated and supports scheduled data scraping to ensure data timeliness and accuracy.

1.4 Data Storage

After the data integration is completed, the complete dataset (including service station basic information and corresponding fuel price data) is saved as a CSV file at the path ‘data.csv’. The storage process uses the ‘to_csv’ method from the pandas library, overwriting existing files to ensure that only the most recent data is retained after each run. This CSV file contains essential station information (such as station ID, brand, address, geographic coordinates) as well as fuel prices for various types and their most recent update times. Prior to storage, data fields are merged and structure is standardized to ensure the generated CSV file is valid and easy for subsequent reading and visualization.

The flowchart of the overall data crawling, processing and storage is as follows:



2.Publishing & Subscribing

2.1 Data Publishing via MQTT

Step 1: Connect to the MQTT Broker

This project uses the paho-mqtt library to connect the client to the specified MQTT Broker:

Broker ip: "172.17.34.107"

Broker port: 1883

Topic: "COMP5339/530446891"

The client will maintain a KeepAlive connection after connecting to ensure that messages can be sent stably. The client publishes the gas station information obtained in real time from the Fuel API to the specified Topic for the streamlit module to subscribe to.

Step 2: Publish data

We read the previously stored csv and convert each line of records into a JSON format string. Next, use the publish() method to publish each record to the topic: "COMP5339/530446891". To avoid excessive pressure on the MQTT Broker by sending a large amount of data in a short period of time, pause for 0.1 seconds after each release to control the sending rate. After all records are published, pause for 60 seconds before starting the next round of crawling and pushing.

2.2 Data Subscribing

Step 1: Initialize the message queue cache

The Streamlit module receives and displays data by subscribing to MQTT Topic.

Use queue.Queue to implement message caching to ensure that Streamlit does not miss received messages.

Step 2: Connect to MQTT Broker and define callback

Broker ip: "172.17.34.107"

Broker port: 1883

Topic: "COMP5339/530446891"

Define callback function on_connect: Automatically subscribe to the target topic when the connection is successful.

Define callback function on_message: After receiving the MQTT message, decode it into JSON format.

Then store the parsed message in the message queue. If the JSON format parsing fails, ignore the message.

In Streamlit, st.session_state is used to retain variable states after page refreshes or interactions. We use state to maintain the following key content:

state["stations"]: All gas station data received.

state["filter_brand"]: Brand filter selected by the user in the sidebar.

state["filter_fuel"]: Fuel filter selected by the user in the sidebar.

Using session_state allows users to filter, drag the map, and other operations without losing previously received data.

Step 3: Process messages in the queue

Save the message queue as the rec variable and traverse the records in the message queue.

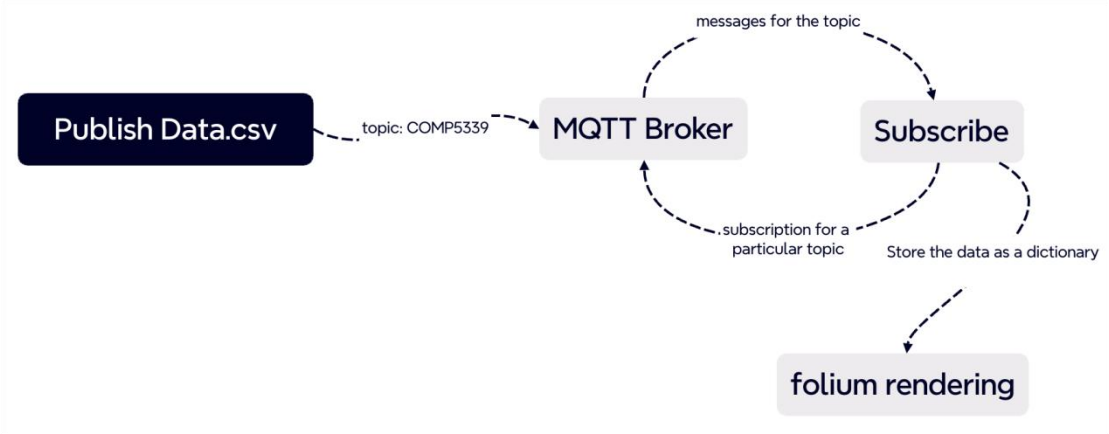
Each record is a JSON string sent by MQTT, where fuel_price and fuel_update_time are dictionaries of string type, so ast.literal_eval() is used to restore them to Python dictionary objects.

The parsed fuels and update_time fields represent all the fuel prices and corresponding update times of the gas station respectively.

Use the gas station unique code code as the dictionary key to insert or update the station information into state["stations"].

Finally, all gas station data are saved in state["stations"] to provide data support for subsequent screening and map display.

The overall data upload and reception process is as follows:



3. Dynamic Visualisation Module

3.1 Module Overview

This module is responsible for presenting the incoming MQTT fuel price data to the interactive map interface. It consists of three parts: dynamic map rendering, interactive marking and fuel type selector. The following will introduce the design, implementation details and challenges encountered during the development process of each part.

3.2 Fully Dynamic Rendering Map

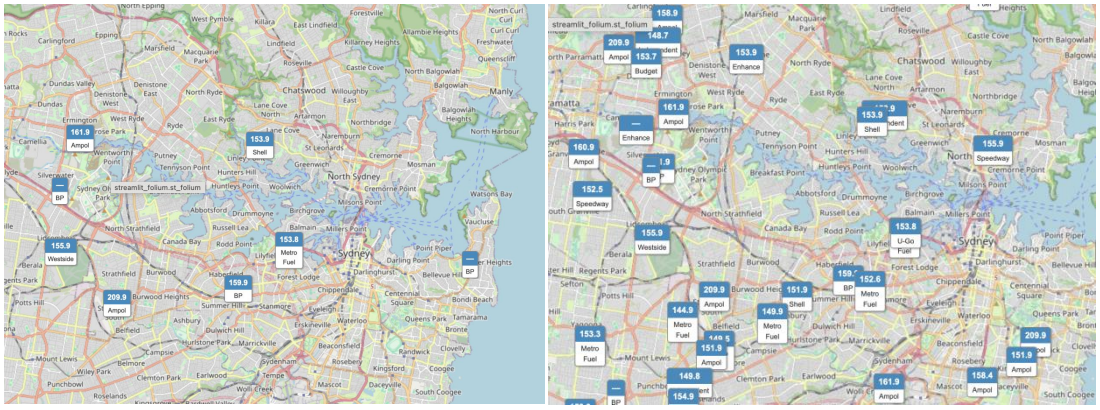


Figure 1. Live Updating Station Map

Function description: This module uses geographical location markers to dynamically render gas station data onto an interactive map. When initializing the map, set a fixed center position (Sydney) and the default oil type to U91. Implementation method: We use folium.Map () to create the basic map object and combine folium.Marker () to dynamically present the gas station. And use FeatureGroup to control layer rendering and updates.

3.3 Interactive Markers

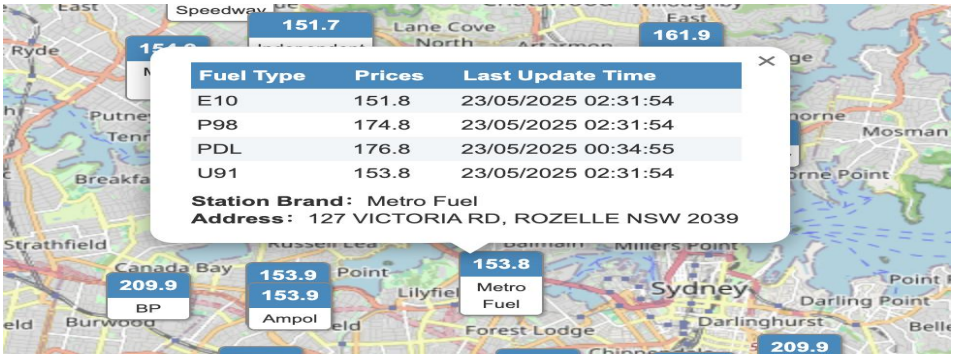


Figure 2. Interactive Fuel Station Popup with Price and Update Time

Function description: This module adds interactive map markers to each gas station, enabling users to click the icon to view detailed information about the site.

The pop-up content includes:name of the gas station, Geographical location of the gas station and Information on various types of fuel and their corresponding prices or status.

Implementation method: Map markers are created using folium.Marker, while information Windows (popup) are constructed using structured HTML strings. We extract data from each site record rec and dynamically concatenate it into HTML content. A highly visualized and structurally flexible map marking system has been realized through folium.Marker. Combined with the two interaction methods of Tooltip and Popup, users can browse the detailed information of each gas station quickly and conveniently.

3.4 Fuel Type Selector

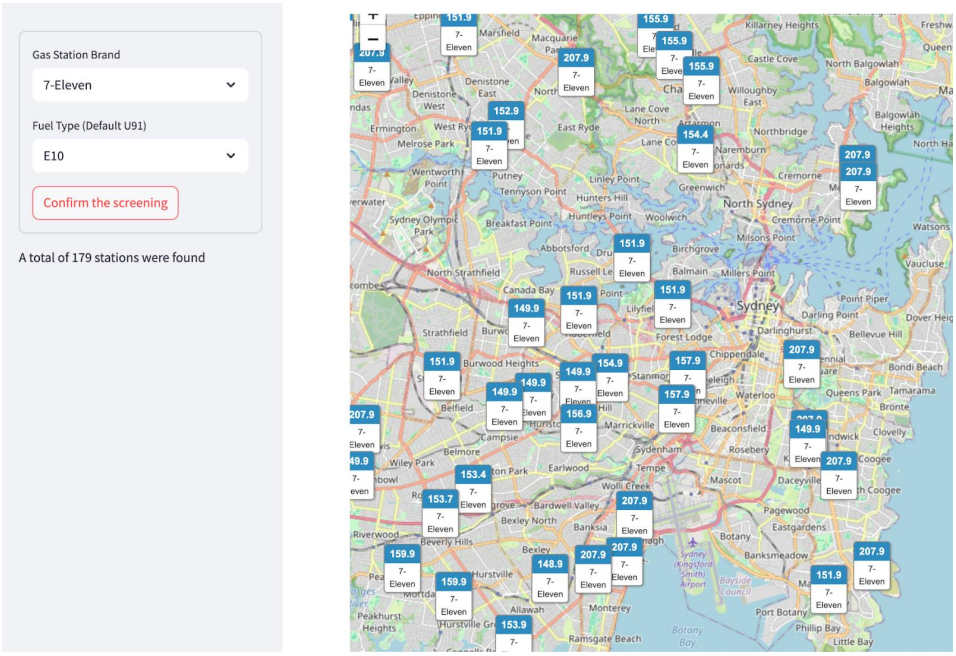


Figure 3. Map Filtering by Fuel Type and Station Brand

Function description: This module is used to control the information of gas stations displayed on the map, mainly based on the following filtering conditions:

1. Fuel Type
2. Gas station brand

By filtering the data, visual distractions on the map can be effectively reduced, loading performance can be improved, and users can focus on the fuel types and brands they are interested in.

4. Identified Issues and Optimization Directions

Map flickering issue (frequent refreshing). In the initial version, the entire map was redrawn every time an MQTT message was received, resulting in severe flickering and affecting the user experience. To solve this problem, we introduce a manual confirmation filter button. The overall refresh will only be performed when the user confirms the filtering. This significantly reduces the number of redrawings and improves the interface stability.

5. Conclusion and Discussion

This research paper retrieves up-to-date fuel prices and station details from the NSW Fuel API and uses integration of api based data retrieval, data cleansing and integration, MQTT-based messaging and dynamic visualization. After processing this data efficiently, it was

concluded that real-time visualization requires not only real-time updated data but also optimization of Streamlit to ensure rendering stability.

QoS levels could be added in future studies of the project to prevent information loss and thus increase data reliability. Additionally, the front-end data can be synchronized and optimized by using a real-time updating front-end map component. Finally, for the front-end, functions such as the user's current location and the user's distance from the gas station can be added to increase the ability of map interaction.

6. Contribution Report

Nan Chen: Completed the Interactive Markers and Fuel Type Selector

Lang Xu: Completed the model overview and fully dynamic rendering map

Fuhai Liang: Participate code writing and debugging. Complete the report writing of the first and second parts

Qiuwen Li: Completed the data clean and data processing.

Bowen Bai: Completed the section of Data Publishing and Subscribing by MQTT and decoding JSON data to state.

Acknowledgement

I acknowledge the use of <ChatGPT (https://chat.openai.com/)> to <enhance readability and structure of the article>.